

# EC4219: Software Engineering

## Lecture 9 — Program Verification (4) *Invariant Inference*

Sunbeom So  
2025 Spring

# Invariant Inference

- The inductive assertion method can be used only when proper loop invariants are given. However, manually writing invariants is hard and tedious.
- This lecture aims to explore a simple yet useful algorithm for automatically generating conjunctive invariants, called **Houdini**.

# Hounini Overview

- Originally proposed as annotation assistant for ESC/Java (Extended Static Checker for Java).
- Named after magician Harry Houdini.
- Based on **Guess-and-Check**: guess some annotations, and check if they are correct.
  - ▶ Generally applicable to inference of any types of invariants (loop invariants, function specifications, etc).
  - ▶ The annotations produced by Houdini are **sound**. That is, inferred invariants are true inductive invariants.
  - ▶ However, it is **incomplete**. The synthesized annotations may not be sufficient to prove property.

cf) In the context of search algorithms, an algorithm is said to be sound iff every returned solution is correct, and complete iff the algorithm guarantees to eventually find a solution (if any).

# Step 1: Guess Invariants

Many different techniques for guessing invariants.

- Collect candidates from source code based on heuristics
  - ▶ Expressions of the form  $v_1 \text{ op } v_2$  or  $v_1 \text{ op } c$ , where  $v_1$  and  $v_2$  are variables used in source code and  $c$  is an “interesting” constant.
- Use dynamic analysis (Daikon approach)
  - ▶ Employ facts while running the program.
- All these techniques are heuristics in nature. Their effectiveness can differ depending on application domains.

## Step 2: Check Invariants

The checker only throws out candidate annotations that are refuted by the verifier.

- A candidate loop invariant  $i \in I$  is refuted if
  - 1 it is not implied by the loop precondition
  - 2 it is not preserved by the loop body
- A candidate function precondition  $p \in P$  is refuted if it does not hold at the function's call-site.
- A candidate function postcondition  $q \in Q$  is refuted if  $\{P\} S \{q\}$  is invalid ( $S$  is the function body).

# Pseudo Code of Houdini

Suppose  $P$  is a program containing a single loop.

---

## Algorithm 1 Houdini

---

**Input:** A program  $P$  to verify

**Output:** A conjunctive invariant  $A$

```
1:  $A_0 \leftarrow$  enumerate speculated invariants
2:  $A \leftarrow A_0$ 
3: while true do
4:   refuted  $\leftarrow$  Verify( $P, A$ )
5:   if refuted =  $\emptyset$  then
6:     return  $A$ 
7:    $A \leftarrow A \setminus \textit{refuted}$ 
```

---

- The algorithm returns the conjunctive invariant  $I_A = \bigwedge_{b_i \in A} b_i$ .
- **Termination:** Terminates after at most  $|A_0|$  iterations.
- **Soundness:** Upon termination, annotations in  $A$  are true invariants.

## Example: Finding Loop Invariants

Consider the simple code below.

```
1  i := 0;
2  j := -1;
3  while (i<1000) {
4      j := i;
5      i := i+1;
6  }
```

Suppose  $A_0 = \{I_1 : i \geq 0, I_2 : i = j, I_3 : i < 1000, I_4 : i \leq 1000\}$ .  
Compute the inductive invariant.

- The candidate  $I_2$  is refuted because it is not implied by the precondition (the following is invalid):

$$\{true\} L1; L2 \{I_2\}$$

- The candidate  $I_3$  is also refuted because the following is invalid:

$$\{I_1 \wedge I_3 \wedge I_4 \wedge i < 1000\} L4; L5 \{I_3\}$$

# Property of Houdini Algorithm

Given a set of candidate loop invariants, Houdini finds the largest subset that is inductive (i.e., the strongest inductive invariant). Why?



# Property of Houdini Algorithm

Given a set of candidate loop invariants, Houdini finds the largest subset that is inductive (i.e., the strongest inductive invariant). Why?

Proof by contradiction:

- Suppose Houdini returns the set  $A$ , but there exists a stronger and inductive invariant  $B$ , i.e.,  $B \supset A$  such that  $\bigwedge_{b_i \in B} b_i$  is inductive.
- This means that the algorithm must have eliminated some  $b_i \in B$  such that  $b_i \notin A$ .
- This happens only when if either (1)  $Pre \rightarrow b_i$  is invalid or (2)  $\{I_B \wedge C\} Body \{b_i\}$  is invalid.
- But neither option is possible since  $B$  is inductive according to our assumption (contradiction!).

# Pseudo Code of Houdini (Generalized)

Let  $Label$  be the location set of annotations.

---

## Algorithm 2 Houdini

---

**Input:** A program  $P$  to verify

**Output:** A conjunctive invariant  $A$

1:  $\mu \leftarrow$  enumerate speculated invariants

$\triangleright \mu : Label \rightarrow \mathcal{P}(\mathbf{FOL})$

2: **while**  $true$  **do**

3:      $refuted \leftarrow \mathbf{Verify}(P, \mu)$

4:     **if**  $refuted = \emptyset$  **then**

5:         **return**  $\mu$

6:     **for each**  $(l, \phi) \in refuted$  **do**

7:          $\mu[l] \leftarrow \mu[l] \setminus \{\phi\}$

---

# Houdini for Function Specifications

- Houdini is not limited to inferring loop invariants, and it can be used to infer function specifications.
- Suppose we have a candidate set of preconditions ( $P$ ) and postconditions ( $Q$ ), and initialize preconditions and postconditions of every function with  $P$  and  $Q$ , respectively.
- When analyzing a function  $F$ :
  - ▶ If verification fails due to the callee's precondition  $p$ , remove  $p$  from the callee's precondition set.
  - ▶ If verification fails because some postcondition  $q$  could not be established, remove  $q$  from the  $F$ 's postcondition set  $Q$ .

## Example: Finding Function Specifications

```
1  main ( ) { foo (5,0); }
2
3  foo (x, y) {
4      if (x<=0) z := y; else z := bar(x,y);
5      return z;
6  }
7
8  bar (x,y) { x := x-1; y := y+1; return foo(x,y); }
```

Suppose  $P = \{P_1 : x \geq 0, P_2 : y \geq 0, P_3 : x = y, P_4 : x > 0\}$ .  
Suppose also  $Q = \{Q_1 : rv \geq 0, rv = 0\}$ . Find function specifications for foo and bar.

## Example: Finding Function Specifications

```
1 main ( ) { foo (5,0); }
2 foo (x, y) {
3   if (x<=0) z := y; else z := bar(x,y);
4   return z;
5 }
6 bar (x,y) { x := x-1; y := y+1; return foo(x,y); }
```

- When analyzing main, we remove  $P_3 : x = y$  for foo.
- When analyzing foo, we remove  $P_3 : x = y$  for bar because `assert(x=y)` fails at bar's callsite.
- When analyzing foo, we remove  $Q_2 : rv = 0$  for foo because `assert(rv=0)` fails ( $rv = 5$ ).

## Example: Finding Function Specifications

```
1  main ( ) { foo (5,0); }
2  foo (x, y) {
3      if (x<=0) z := y; else z := bar(x,y);
4      return z;
5  }
6  bar (x,y) { x := x-1; y := y+1; return foo(x,y); }
```

- When analyzing bar, we remove  $P_4 : x > 0$  for foo.
- When analyzing bar, we remove  $Q_2 : rv = 0$  for bar.
- Iterate the same process, and nothing is refuted.
- The inferred function specification for foo:

$$P = \{x \geq 0 \wedge y \geq 0\} \text{ and } Q = \{rv \geq 0\}$$

- The inferred function specification for bar:

$$P = \{x \geq 0 \wedge y \geq 0\} \text{ and } Q = \{rv \geq 0\}$$

# Summary

Houdini algorithm: a simple approach for automatically inferring conjunctive invariants:

- Pros: general applicability, easy to implement
- Cons: infer conjunctive invariants only, not property-directed (does not guarantee that the inferred invariants are useful for verifying property)

Finding invariants still remains an active research area (probabilistic reasoning, domain-specific refinement, etc).